unspecified behaviour is not banned outright in programs. According to the C11 standard, the order of evaluation of sub-expressions, the order of evaluation of function arguments, the values of any padding bits in integer representations, the order in which # and ## operations are evaluated during macro substitution, etc, are examples for unspecified behaviour in C.

I have referred to the document numbered N1570 of the draft ISO/IEC 9899:201x dated April 12, 2011 for the C11 programming language standard and the document numbered N4140 dated October 7, 2014 for the C++14 programming language standard. I encourage you to refer to these documents for any clarifications and further details. Finally, regarding unspecified behaviour, let us say the moral of the story is "all programmers need not be afraid of unspecified behaviour, but only those who want their programs to have portability need to worry." But, unfortunately, programmers who program for just one compiler and one system are often considered as people not worth their salt. So it is better to think about portability even if you are writing a program just for your amusement.

Now, let us work with some C programs to better understand unspecified behaviour. I have compiled some programs with code fragments exhibiting unspecified behaviour with a number of compilers to check whether the output obtained is the same or different with different compilers. The test results were mixed. For some programs, all the compilers showed the same output, and for some other programs the output depended on the compiler even when executed on a single system. I have used three different C compilers and two different C++ compilers in Linux to test whether a piece of code with the so-called unspecified behaviour acts surprisingly or not. The three C compilers used for testing are gcc, tcc, and Clang. The two C++ compilers used for testing are g++ and Clang++. The GNU Compiler Collection (GCC) provides the C compiler gcc and the C++ compiler g++. The LLVM compiler infrastructure provides the C compiler Clang and the C++ compiler Clang++. The Tiny C Compiler (tcc) is a C compiler created by Fabrice Bellard.

The first unspecified behaviour we are going to explore in detail is the order of evaluation of sub-expressions. First of all, what does the evaluation order of a sub-expression mean? Consider the C statement 'a=b+c;' where a, b, and c are integer variables. It is not specified in the C or C++ standards whether the value of the variable b or the variable c is fetched first to perform the addition operation. In this particular case, the output does not depend on the evaluation order, but this may not be the case always. Consider the program named *pgm1.c* shown below as an example, in which a change in the evaluation order changes the output. The program *pgm1.c* and all the other programs discussed in this article can be downloaded from *opensourceforu.com/article_source_code/Feb19unspecifiedbehaviourinC.zip*.

```
#include<stdio.h>

int x=100;

int left( )
{
     x++;
     return x;
}

int right( )
{
     x=1;
     return x;
}

int main( )
{
     int y=left( )+right( );
     printf("\nSum = %d\n\n",y);
}
```

In the program *pgm1.c*, the line of code causing unspecified behaviour is *int y=left( )+right( );* due to the unspecified evaluation order of sub-expressions. Here, the output depends on the evaluation order because both the functions *left( )* and *right( )* are modifying the same global variable *x*. If the function *left( )* is called first then the value of the global variable *x* will become 101, because the statement *x++;* gets evaluated first and therefore the function *left( )* returns the value 101. Later, when the function *right( )* gets called, the value of the global variable *x* becomes 1 because of the statement *x=1;* therefore the function *right( )* returns the value 1. Thus, in this case, the output will be 102.

Let's suppose the function *right( )* is called first, then the value of the global variable *x* will become 1, because the statement *x=1;* gets evaluated first and therefore the function *right( )* returns the value 1. Later, when the function *left( )* gets called, the value of the global variable *x* becomes 2 because of the statement *x++;* and therefore the function *left( )* returns the value 2. Thus, in this second case, the output will be 3. So, in this program, the output depends on the evaluation order. But when I tested the program with the three different C compilers mentioned earlier, all of them behaved in the same way by calling the function *left( )* first and the function *right( )* second, and thus producing the output 102. Figure 1 shows the output from the gcc, Clang, and tcc compilers when the program *pgm1.c* is compiled.

I have also renamed the program *pgm1.c* as *pgm1.cc* and tested it with the C++ compilers g++ and Clang++, but the output again was 102 because the function *left( )* was called first and the function *right( )* was called second. But we have to keep two things in mind regarding the behaviour of this program. First, even though all the compilers tested here gave